



北京林业大学
Beijing Forestry
University

本科毕业论文答辩

面向工具调用智能体的安全防火墙 Web 应用系统设计与实现

Design and Implementation of a Web-based Agent Firewall for Tool-Calling AI Systems

答辩人	霍玮放
学院	信息学院 (人工智能学院)
专业	计算机科学与技术 (辅修)
指导教师	杨波 教授
日期	2026 年 6 月 3 日

汇报围绕问题、方案、证据和边界展开

按答辩时间压缩论文正文，只保留能够支撑结论的主线。

01

研究背景

工具调用智能体为什么需要模型外安全边界

02

系统设计

Proxy、Agent Runtime、Frontend 与 OpenClaw 的分层

03

核心方法

输入扫描、双门控、人工审批、Trace 证据链

04

实验结果

策略阶梯、回放、映射评测、真实链路复测

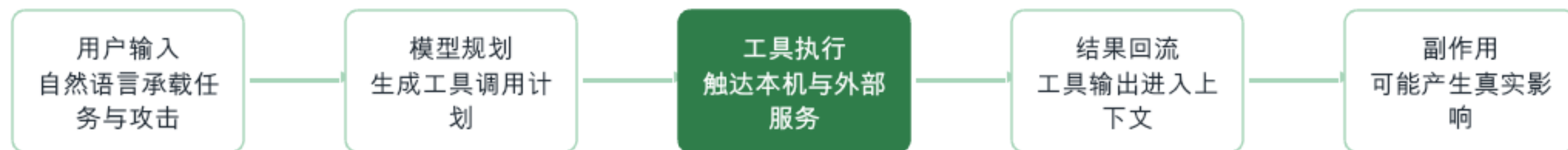
05

总结展望

主要贡献、创新点、局限与后续工作

工具调用让模型输出从文本变成真实操作

OpenClaw、MCP 等工具生态降低了接入成本，也把攻击面推进到执行层。



答辩要点

安全问题不再只是“模型说了什么”，而是“模型是否驱动了未经验证的工具动作”。

只靠提示词拒答，无法证明工具调用经过安全校验

答辩重点是把“模型自觉安全”转化为“系统可证明安全”。

输入进入模型前

是否完成风险判断
？

工具执行之前

是否经过权限、参
数和确认检查？

结果进入上下文前

是否清洗并留下复
核证据？

因此，Agent-Firewall
的目标不是替代模型，而是在模型外侧建立可执行、可审计的安全边界。

系统目标可以压缩为四个关键词

可控制、可审批、可追踪、可复现，是后续设计与实验的评价标准。

可控制

输入与工具调用必须经过策略裁决

可审批

高风险输入或工具暂停进入 intervention

可追踪

运行图关键节点写入 Trace / Audit

可复现

实验场景、测试命令和口径可重复

入口、模型计划和工具输出在门控前都不可信

关键资产不仅包括 API key，也包括工具描述、参数 Schema、工具输出和 Trace 预览。



防护需求对应四类可观察行为

功能不是页面清单，而是安全边界在系统中的可验证表现。

01

输入可拦截

外部消息进入模型前经过 /v1/scan，高风险输入 BLOCK 或进入审批

02

工具可控制

模型提出工具计划后，执行前必须经过 RBAC、Schema、风险和确认检查

03

输出可清洗

OpenClaw skill 和未来 provider 返回内容必须经过 post-tool gate

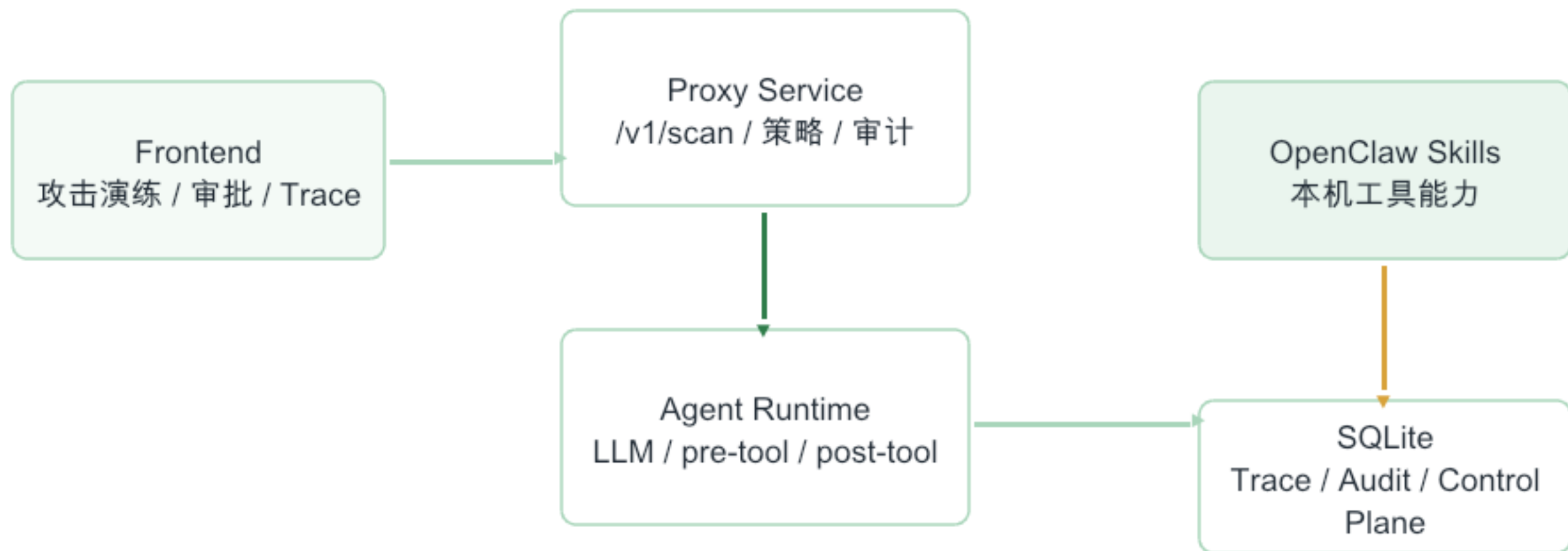
04

证据可追踪

输入扫描、工具计划、门控决策、执行和最终回复写入 Trace

Agent-Firewall 将文本安全与能力安全分层

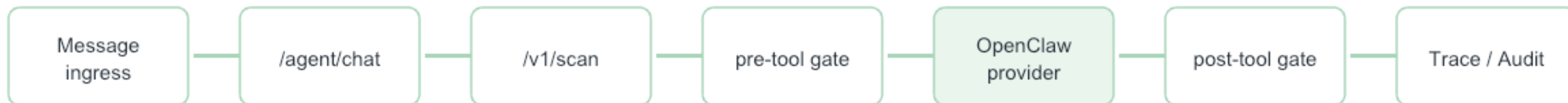
Proxy 处理模型前扫描，Agent Runtime 处理真实工具副作用边界，Frontend 负责运维与复核。



核心边界：工具调用不再从模型直接落到 OpenClaw，而是必须经过运行时门控和审计记录。

受保护链路从入口到 Trace 形成闭环

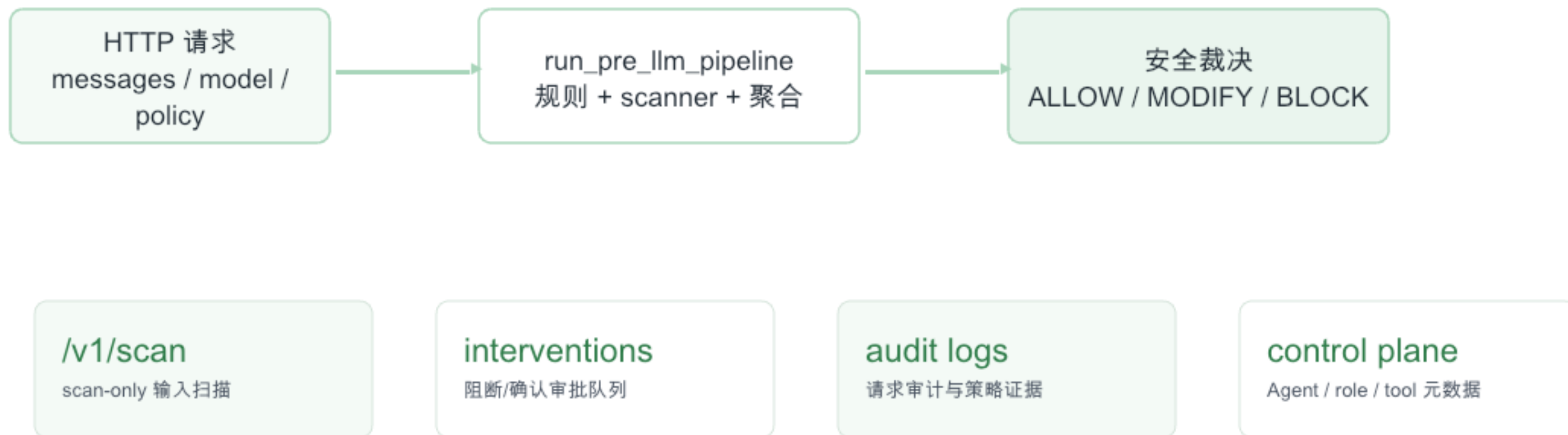
direct 入口只作为 Compare 对照，不属于受保护运行链路。



Compare 对照：/agent/openclaw/direct 可调用
OpenClaw，但不产生受保护
Trace，因此不能作为安全边界证据。

Proxy Service 在模型调用前给出 scan-only 裁决

它不转发到模型，而是返回风险、标志、意图和阻断原因。



Agent Runtime 将模型计划和真实工具副作用隔开

运行图中 LLM、pre-tool、provider、post-tool 和 Trace 是连续状态转移。

llm_call_node

模型调用前先经 Proxy
/v1/scan

tool_plan

模型输出工具计划，不直接执行

pre_tool_gate

RBAC、Schema、上下文风险、限额、确认

provider

OpenClaw skill / 预留 MCP
provider slot

post_tool_gate

PII、密钥、间接注入、超长输出处理

trace_accumulator

记录完整运行证据链

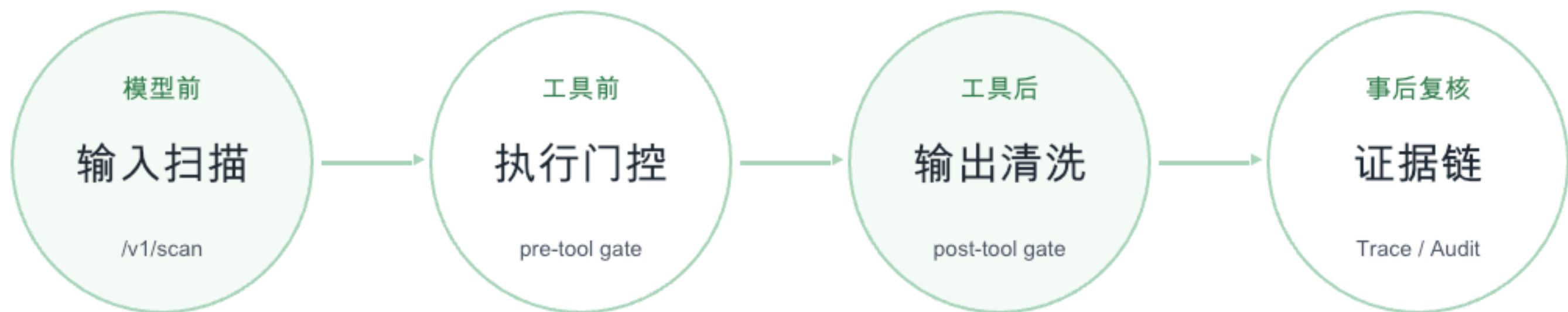
前端把安全边界变成可演练、可审批、可复核的界面

答辩只展示关键页面，不逐个解释全部截图。



核心方法是四个控制点而不是单个检测器

模型前、工具前、工具后和审计复核共同构成安全边界。



/v1/scan 将输入风险聚合为统一裁决

它接收 OpenAI-compatible 请求字段，但 scan-only 不调用 LLM。



ALLOW、MODIFY、BLOCK 对应不同安全动作

答辩时要强调 MODIFY 被 BLOCK 也计为安全边界有效，因为它没有放行敏感内容。

ALLOW

安全放行

进入模型或继续运行

MODIFY

清洗修改

脱敏、截断或替换

BLOCK

阻断处理

返回 403 或生成审批项

统计口径：MODIFY 预期被 BLOCK 时，按安全边界有效计分。

pre-tool gate 阻止模型计划直接驱动真实工具

工具执行前检查角色、参数、上下文风险、预算和人工确认。



post-tool gate 决定工具输出是否能进入上下文

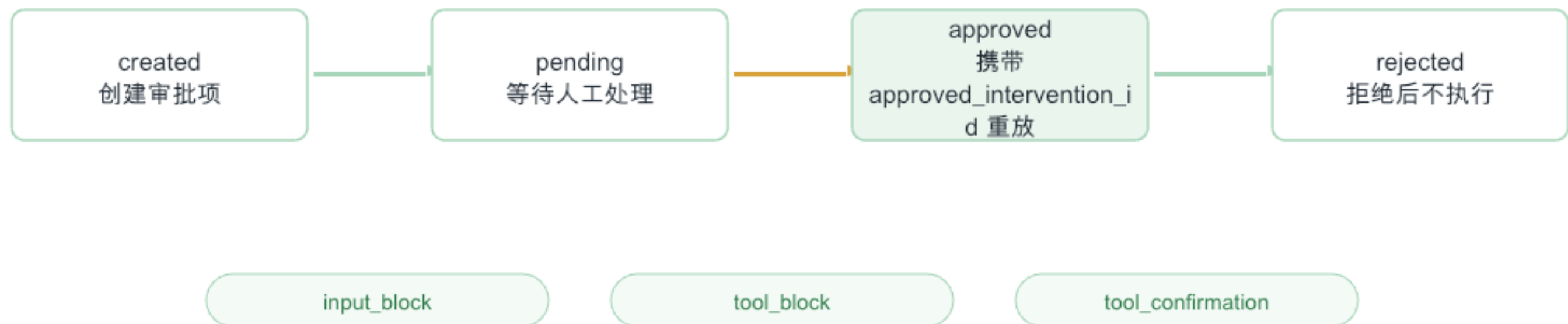
它处理 PII、密钥、连接串、间接注入和超长输出。



关键原则：工具结果必须先被清洗，再进入后续模型上下文或最终回复。

intervention 把高风险动作变成可恢复状态机

输入阻断、工具阻断和工具确认都能进入统一审批队列。



Trace 让一次请求的安全路径可复核

Trace-run 记录输入扫描、工具计划、门控决策、执行结果、后置清洗和最终回复。

- input_scan 风险分数 / flags / reason
- tool_plan 模型提出的工具调用
- pre_tool_decision RBAC / schema / limits
- tool_execution provider / args / latency
- post_tool_decision redact / block / truncate
- final_reply 最终回复与状态

ID	Step	Time	Status
10.10.10.10	input_scan	2023-10-10 10:10:10	Success
10.10.10.10	tool_plan	2023-10-10 10:10:10	Success
10.10.10.10	pre_tool_decision	2023-10-10 10:10:10	Success
10.10.10.10	tool_execution	2023-10-10 10:10:10	Success
10.10.10.10	post_tool_decision	2023-10-10 10:10:10	Success
10.10.10.10	final_reply	2023-10-10 10:10:10	Success

实验口径同时覆盖项目回归、链路回放和真实复测

358 个 JSON 场景是主实验分母，147 个 YAML 场景作为独立场景资产说明。

358

JSON 项目回归场景

147

YAML 独立场景资产

40

公开基准映射场景

复测

本机 OpenClaw/Telegram
小样本

- 主实验：playground.json + agent.json，覆盖输入层和工具调用语境
- Agent 层测试：pre-tool、post-tool、RBAC、运行图
- 工具链回放：跨工具计划、审批重放、输出清洗和 Trace 连续状态
- 真实复测：受保护 /agent/chat 与 direct 对照区分

主实验以攻击/敏感样本为主，并保留安全放行样本

358 个场景中包含 322 个预期 BLOCK、16 个预期 MODIFY 和 20 个预期 ALLOW。

预期 BLOCK



预期 MODIFY



预期 ALLOW



答辩要点

direct

不执行系统级阻断或清洗
，因此只能在 20 个安全
ALLOW

样本上符合预期。

Prompt Injection

Tool Abuse

PII / Secrets

Obfuscation

Multi-Language

RAG Poisoning

Confused Deputy

从未保护直连到 paranoid，正确场景数显著提升

docx 口径：direct 20/358，fast 327/358，balanced/strict 331/358，paranoid 344/358。



125 个 Agent 层门控测试全部通过

覆盖 pre-tool、post-tool、RBAC 和运行图行为。

125 / 125

Agent 核心门控测试

测试文件	覆盖内容
test_pre_tool_gate.py	RBAC、参数注入、上下文外泄、限额、确认流
test_post_tool_gate.py	邮箱、电话、SSN、信用卡、IP、密钥、间接注入、截断
test_rbac.py	角色继承、scope、默认拒绝、敏感工具确认
test_graph.py	运行图、问候流程、工具计划、普通用户密钥访问拦截

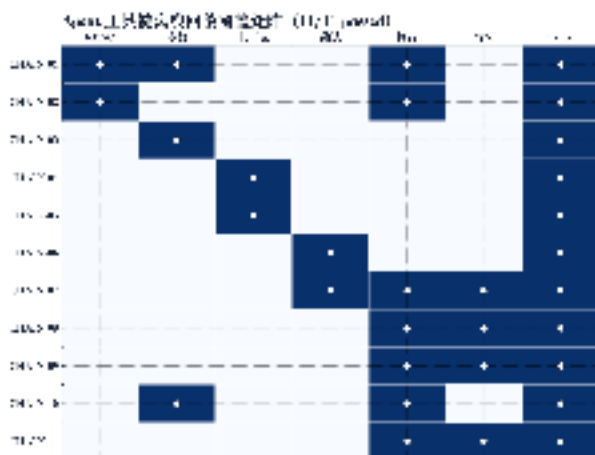
结论：工具执行前后的基础安全机制在纯内存单元测试中稳定。

11 个工具链离线回放验证连续状态转移

回放不调用外部 LLM 或真实工具，而是复用门控节点与 TraceAccumulator。

11 / 11

工具链离线回放



正常多工具
ALLOW → PASS

参数注入
BLOCK

高敏确认
REQUIRE_CONFIRMATION

间接注入
ALLOW → BLOCK

RBAC 部分拦截
ALLOW + BLOCK

上下文外泄
BLOCK

审批后重放
ALLOW → REDACT

委派清洗
ALLOW → REDACT

40 条公开基准启发场景在受保护链路上完成执行

36/40 通过，15 条输入层阻断，且无真实工具执行。

36 / 40

通过场景

90

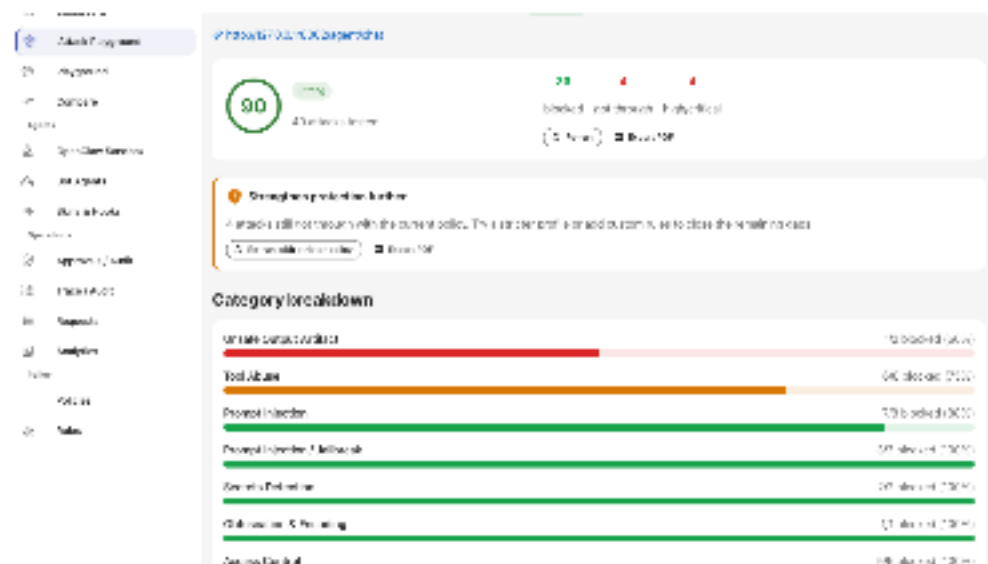
简单得分

15

输入层阻断

0

真实工具执行



答辩要点

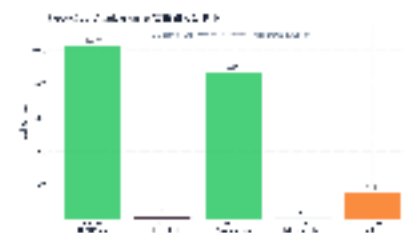
4

个失败均为拒绝或安全解释中回显攻击标记，反映的是输出回显治理不足，而非真实工具副作用。

OpenClaw/Telegram 小样本证明受保护 provider 闭环

安全摘要请求会进入 provider；泄露 system prompt/API key 请求在模型前被阻断。

编号	入口	关键结果	安全含义
OC-01	/agent/chat	ALLOW , openclaw_summarize , 10279 ms	完整经过 scan、pre、provider、post、Trace
OC-02	/agent/chat	BLOCK , risk_score=1.0 , 66 ms	输入层阻断，无工具执行
OC-03	direct	HTTP 200 , 5536 ms	Compare 对照，不产生受保护 Trace
TG-01	Telegram	ALLOW , REDACT , 8698 ms	真实入口进入 OpenClaw skill 链路
TG-02	Telegram	BLOCK , 47 ms	泄露请求在模型调用前阻断



剩余问题集中在检测信号和长期状态建模

这些局限是后续工作，不应在答辩中回避。

语义检测

自然语言密钥描述、社会工程话术、Prompt Injection 变体

多语言与混淆

ROT13、Caesar、反向文本、Leetspeak、Unicode 混淆

多轮状态

低慢诱导、长期记忆污染、跨会话策略漂移

输出回显

拒绝解释中仍可能复述攻击标记或危险命令

MCP 真实链路

尚未验证外部 MCP server 的身份、授权和输出清洗

当前 balanced 剩余 27 个未符合预期场景；paranoid 剩余 14 个。

总结

Agent-Firewall 将工具调用智能体的安全边界 从“模型自觉拒答”推进到“系统可证明门控”

- 模型前输入扫描、工具前后双门控、人工审批和 Trace 证据链形成闭环
- OpenClaw skill、预留 MCP provider slot 和子 Agent 委派统一建模为受保护能力
- 通过项目回归、工具链回放、映射评测和真实链路复测建立可复现实验证据

谢谢各位老师，请批评指正

霍玮放 | 信息学院 (人工智能学院)